

```

import os
from typing import Dict, Optional
from classy import Class
import numpy as np
import pycuba
import hitomipy
import initial

class ComputePowerBiSpectrum():
    r"""Computation of the tree level, the FoG counterterm, the shot noise and the primordial non gaussianity contribution of the bispectrum, as well as the tree level, the one loop correction, and EFT counterterms of the power spectrum, either split into kernels or not. All computation of the bispectrum are done using the tripolar spherical harmonic decomposition formalised in ref. [1]_.
    .. [1] Sugiyama N. et al., 2019. **MNRAS** 484(1), 364-384.
    [arXiv: '1803.02132 <https://arxiv.org/abs/1803.02132>']
    The bispectrum is here characterised by two wavenumbers :math:`k_1` and :math:`k_2` and three multipoles :math:`l_1`, :math:`l_2`, :math:`L`, the latest states for the LOS anisotropies.

    Parameters
    -----
    params_cosmo : dict
        Cosmological parameters to compute linear power spectrum with Class.
    z : float
        Redshift.
    diag : bool, optional
        If 'True' (default), model computed on the diagonal part of the bispectrum.
    Attributes
    -----
    Da : float
        Angular distance at the given cosmology and redshift.
    H : float
        Hubble parameter at the given cosmology and redshift.
    K : dict
        Result of the computation stored with parameters used (wavenumbers, multipoles, kernels).
    """

    def __init__(self, params_cosmo: Dict, z: float, diag: Optional[bool] = True) -> None:
        if not isinstance(params_cosmo, dict):
            raise ValueError("params_cosmo must be a dictionary")
        if not isinstance(z, (float, int)):
            raise ValueError("z must be a float or integer")
        if not isinstance(diag, bool):
            raise ValueError("diag must be a boolean")

        self.params_cosmo = params_cosmo
        self.z = z
        self.diag = diag

        # initialize instance variables
        self.initial_cosmo = None
        self.k_in = None
        self.pk_in = None
        self.pk_in_no_wiggle = None
        self.f = None
        self.rs_drag = None
        self.sigma8_norm = None
        self.Da = None
        self.H = None

    def __str__(self) -> str:
        return f"ComputePowerBiSpectrum(params_cosmo={self.params_cosmo}, z={self.z}, diag={self.diag})"

    def __repr__(self) -> str:

```

```

    return self.__str__()

    def initial_power_spectrum(self, cosmo: Optional[Class] = None, real: bool = False, a
    perp=0, apar=0) -> None:
        """Computation of the linear power spectrum, the no wiggle part of it
        and derived cosmological parameters.
        -----
        cosmo : :class:`classy.Class`, optional
            If 'False' (default), Class is instanciate with params_cosmo.
        """

        #if cosmo is None:
        #    cosmo = Class()
        #    dict_cosmo = self.params_cosmo
        #    dict_cosmo.update({'z_pk':self.z, 'non linear':'PT', 'IR resummation':'Yes', 'B
ias tracers':'Yes', 'cb':'Yes',
        #                                'RSD':'Yes', 'AP':'No', 'PNG':'No',
        #                                'output': 'mPk', 'z_max_pk': 3., 'P_k_max_h/Mpc':
50.})
        #    cosmo.set(self.params_cosmo)
        #    cosmo.compute()

        print('in initial power spectrum')
        self.initial_cosmo = initial.InputPowerSpectrum(self.z, cosmo, params_fid=self.pa
rams_cosmo)
        self.initial_cosmo.calcMatterPowerSpectrum()
        self.k_in, self.pk_in = self.initial_cosmo.getMatterPowerSpectrum()
        self.pk_in_no_wiggle = self.initial_cosmo.getNoWigglePowerSpectrum()

        self.f = 0 if real else self.initial_cosmo.getGrowthRate()
        self.rs_drag = cosmo.rs_drag()
        self.sigma8_norm = self.initial_cosmo.getSigma8ForNormalization()
        self.Da, self.H = self.initial_cosmo.calcFiducialHubbleAndDiameterDistance()

        self.aperp = aperp
        self.apar = apar

    def kernel_computation_Bk(self, kernel_name, k, ell1, ell2, ELL, integrand='tree', to
_save=False, f_fit=False, fs8norm=True):
        """Call the diagonal or full part of bispectrum model kernel.
        -----
        kernel_name : str
            bias parameters associated to the kernel to compute.
        k : list
            wavevectors at which compute the kernel.
        ell1 : int
            first multipole of the tripOSH decomposition.
        ell2 : int
            second multipole of the tripOSH decomposition.
        ELL : int
            third multipole of the tripOSH decomposition, LOS anisotropies.
        to_save : str, optional
            If 'False' (default), kernel not saved.
        """

        self.kernel_name = kernel_name
        self.k = k
        self.ell1 = ell1
        self.ell2 = ell2
        self.ELL = ELL
        self.fs8norm = fs8norm
        self.f_fit = f_fit

        if self.diag: self.calc_K_diag(integrand=integrand)
        else: self.calc_K_full(integrand=integrand)

        if to_save:
            self.BK['params_cosmo'] = self.params_cosmo
            self.BK['z'] = self.z
            self.save(to_save)

```

```

def calc_K_full(
    self,
    integrand='tree',
    ks=.05,
):
    """Computation of the full bispectrum kernel.
    -----
    integrand : str, optional
        part of the bispectrum model to compute. 'tree' (default),
        'SN' (shot noise) or 'PNG' (primordial non gaussianity).
    ks : float, optional
        Separation scale for IR resummation (default is 0.05), no impact on
        the calculation for physical scales.
    """

    self.integrand = integrand
    (kbin2_out, kbin1_out) = np.meshgrid(self.k, self.k)
    ## flags ##
    output_dict_ini = {
        "kbin1": kbin1_out,
        "kbin2": kbin2_out,
        "K": np.zeros((len(self.k), len(self.k))),
        "ell1": self.ell1,
        "ell2": self.ell2,
        "ELL": self.ELL,
    }

    ## initialization ##
    hitomipy.initializeInputPowerSpectrum_py()

    ## calc. wigner 3j symbols ##
    hitomipy.setWigner3j_py()

    ## read linear power spectrum ##
    hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
    hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

    ## normalization ##
    hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
    h = self.params_cosmo['h']
    Omega_b = self.params_cosmo['omega_b']/h**2
    Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm'])/h**2
    Tcmb = 2.7255
    n_s = self.params_cosmo['n_s']
    hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

    ## IR resummation damping term ##
    self.Sigma2 = hitomipy.Sig2_py(self.rs_drag, ks)
    self.dSigma2 = hitomipy.dSig2_py(self.rs_drag, ks)

    ## compute bispectra ##
    NDIM = 3
    NCOMP = len(self.k)
    if NCOMP > 1024:
        print("# of NCOMP should be <= 1024, otherwise results become zero.")
        return output_dict_ini

    AA = []
    for i in range(NCOMP):
        print("k1 = ", self.k[i], "h/Mpc")
        self.kmag1 = self.k[i]
        AA.append(
            pycuba.Cuhre(
                self.Integrand_K_Bk, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
            )["results"]
        )

    bk_temp = np.zeros((NCOMP, NCOMP))

```

```

for i in range(NCOMP):
    for j in range(NCOMP):
        bk_temp[i, j] = AA[i][j]["integral"]

## finalize parameters ##
hitomipy.finalizeInputPowerSpectrum_py()

#####
(kbin2_out, kbin1_out) = np.meshgrid(self.k, self.k)

## sigma8 normalisation ##
if not self.fs8norm:
    if 'f' in self.kernel_name:
        bk_out = bk_temp/self.f**self.kernel_name.count('f')
    else:
        bk_out = bk_temp
else:
    if self.f_fit:
        bk_temp /= self.f**self.kernel_name.count('f')
    if self.integrand == 'tree': bk_out = bk_temp*self.sigma8_norm**4
    elif self.integrand == 'SN': bk_out = bk_temp*self.sigma8_norm**2
    elif self.integrand == 'PNG': bk_out = bk_temp*self.sigma8_norm**3

## output dict ##
self.BK = {
    "kbin1": kbin1_out,
    "kbin2": kbin2_out,
    "K": bk_out,
    "ell1": self.ell1,
    "ell2": self.ell2,
    "ELL": self.ELL,
    "kernel": self.kernel_name
}

def calc_K_diag(
    self,
    integrand='tree',
    ks=.05,
):
    """Computation of the diagonal part of the bispectrum kernel.
    -----
    integrand : str, optional
        part of the bispectrum model to compute. 'tree' (default),
        'SN' (shot noise) or 'PNG' (primordial non gaussianity).
    ks : float, optional
        Separation scale for IR resummation (default is 0.05), no impact on
        the calculation for physical scales.
    """

    self.integrand = integrand

    ## flags ##
    output_dict_ini = {
        "kbin": self.k,
        "K": np.zeros((len(self.k))),
        "ell1": self.ell1,
        "ell2": self.ell2,
        "ELL": self.ELL,
    }

    ## initialization ##
    hitomipy.initializeInputPowerSpectrum_py()

    ## calc. wigner 3j symbols ##
    hitomipy.setWigner3j_py()

    ## read linear power spectrum ##
    hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
    hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

```

```

    if integrand=='PNG':
        self.initial_cosmo.calcPrimordialPowerSpectrum(self.params_cosmo['ln10^{10}A_
s'],self.params_cosmo['n_s'])
        self.pk_in_primordial = self.initial_cosmo.getPrimordialPowerSpectrum()
        hitomipy.readInputPrimordialPowerSpectrum_py(self.k_in, self.pk_in_primordial
, len(self.k_in))

    ## normalization ##
    hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
    h = self.params_cosmo['h']
    Omega_b = self.params_cosmo['omega_b']/h**2
    Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm'])/h**2
    Tcmb = 2.7255
    n_s = self.params_cosmo['n_s']
    hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

    # IR resummation damping term
    self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks)
    self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks)

    ## compute bispectra ##
    NDIM = 3
    NCOMP = len(self.k)
    if NCOMP > 1024:
        print("# of NCOMP should be <= 1024, otherwise results become zero.")
        return output_dict_ini

    AA = pycuba.Cuhre(
        self.Integrand_K_Bk, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
    )["results"]

    bk_temp = np.zeros((NCOMP))
    for i in range(NCOMP):
        bk_temp[i] = AA[i]["integral"]

    ## finalize parameters ##
    hitomipy.finalizeInputPowerSpectrum_py()

    #####

    ## sigma8 normalisation ##
    if not self.fs8norm:
        if 'f' in self.kernel_name:
            bk_out = bk_temp/self.f**self.kernel_name.count('f')
        else:
            bk_out = bk_temp
    else:
        if self.f_fit:
            bk_temp /= self.f**self.kernel_name.count('f')
        if self.integrand == 'tree': bk_out = bk_temp*self.sigma8_norm**4
        elif self.integrand == 'SN': bk_out = bk_temp*self.sigma8_norm**2
        elif self.integrand == 'PNG': bk_out = bk_temp*self.sigma8_norm**3

    ## output dict ##
    self.BK = {
        "kbin": self.k,
        "K": bk_out,
        "ell1": self.ell1,
        "ell2": self.ell2,
        "ELL": self.ELL,
        "kernel": self.kernel_name
    }

def Integrand_K_Bk(self, ndim, xx, ncomp, ff, userdata):
    """Call of the function to integrate for kernel part.
    -----
    No direct call of this function needed.
    """

    self.xx_in = np.zeros(ndim[0])

```

```
self.ff_out = np.zeros(ncomp[0])

for i in range(ndim[0]):
    self.xx_in[i] = xx[i]

self.ndim = ndim[0]
self.ncomp = ncomp[0]

if self.integrand=='tree':
    if self.diag:
        hitomipy.integrand_B_Kernel_diag_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ell1,
            self.ell2,
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.aperp,
            self.apar,
            self.kernel_name
        )
    else:
        hitomipy.integrand_B_Kernel_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ell1,
            self.ell2,
            self.ELL,
            self.kmag1,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.kernel_name
        )

elif self.integrand=='SN':
    if self.diag:
        hitomipy.integrand_B_Kernel_SN_diag_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ell1,
            self.ell2,
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.aperp,
            self.apar,
            self.kernel_name
        )
    else:
        hitomipy.integrand_B_Kernel_SN_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
```

```

        self.ncomp,
        self.k,
        len(self.k),
        self.ell1,
        self.ell2,
        self.ELL,
        self.kmag1,
        self.f,
        self.Sigma2,
        self.dSigma2,
        self.kernel_name
    )

elif self.integrand=='PNG':
    hitomipy.integrand_B_Kernel_PNG_diag_py(
        self.xx_in,
        self.ndim,
        self.ff_out,
        self.ncomp,
        self.k,
        len(self.k),
        self.ell1,
        self.ell2,
        self.ELL,
        self.f,
        self.Sigma2,
        self.dSigma2,
        self.kernel_name
    )

for i in range(ncomp[0]):
    ff[i] = self.ff_out[i]

return 0

def save(self,to_save):
    """Save the result dictionary of the computation.
    -----
    to_save : str
        name of the file to save the result.
    """

    os.makedirs(os.path.dirname(to_save), exist_ok=True)
    np.save(to_save,self.BK)

def load(self,to_load):
    """Loading of dictionary saved in .npy file.
    -----
    to_load : str
        name of the file to load.
    """

    return np.load(to_load,allow_pickle=True).item()

def calc_B_diag(
    self,
    k, ell1, ell2, ELL,
    alpha_perp=1, alpha_parallel=1, b1=2, b2=0, bG2=0, c1=0, c2=0, knl=.3,
    fnlloc=0, fnlequi=0, fnlortho=0,
    Pshot=0, Bshot=0,
    integrand='tree',
    ks=.05,
    Sigma2=False,
    f=False
):
    """Computation of the diagonal part of the bispectrum.
    -----
    k : list
        waveevenumbers at which compute the kernel.
    ell1 : int

```

```
    first multipole of the tripoSH decomposition.
ell12 : int
    second multipole of the tripoSH decomposition.
ELL : int
    third multipole of the tripoSH decomposition, LOS anisotropies.
alpha_perp : float, optional
    perpendicular AP scaling between cosmo_fid and the cosmology studied
    default is 1.
alpha_parallel : float, optional
    parallel AP scaling between cosmo_fid and the cosmology studied
    default is 1.
b1 : float, optional
    linear galaxy bias (default is 2).
b2 : float, optional
    second order galaxy bias (default is 0).
bG2 : float, optional
    tidal galaxy bias (default is 0).
c1 : float, optional
    FoG first counterterm (default is 0).
c2 : float, optional
    FoG second counterterm (default is 0).
knl : float, optional
    cutoff scale for FoG terms. (default is 0.3).
Pshot : float, optional
    shotnoise term (in power and bispectrum). (default is 0).
Bshot : float, optional
    shotnoise term (in bispectrum). (default is 0).
integrand : str, optional
    part of the bispectrum model to compute. 'tree' (default),
    'SN' (shot noise) or 'PNG' (primordial non gaussianity).
ks : float, optional
    Separation scale for IR resummation (default is 0.05), no impact on
    the calculation for physical scales.
"""

self.k = k
self.ell1 = ell1
self.ell2 = ell2
self.ELL = ELL
self.alpha_perp = alpha_perp
self.alpha_parallel = alpha_parallel
self.b1 = b1
self.b2 = b2
self.bG2 = bG2
self.c1 = c1
self.c2 = c2
self.knl = knl
self.Pshot = Pshot
self.Bshot = Bshot
self.fnllloc = fnllloc
self.fnlequi = fnlequi
self.fnlortho = fnlortho
self.integrand = integrand

if f: self.f = f

## flags ##
output_dict_ini = {
    "kbin": self.k,
    "K": np.zeros((len(self.k))),
    "ell1": self.ell1,
    "ell2": self.ell2,
    "ELL": self.ELL,
}

## initialization ##
hitomipy.initializeInputPowerSpectrum_py()

## calc. wigner 3j symbols ##
hitomipy.setWigner3j_py()
```



```

    ## read linear power spectrum ##
    hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
    hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))
    if integrand=='PNG':
        print(self.params_cosmo)
        if 'ln10^{10}A_s' in self.params_cosmo:
            lnAs = self.params_cosmo['ln10^{10}A_s']
        else:
            lnAs = np.log(1e10*self.params_cosmo['A_s'])
        self.initial_cosmo.calcPrimordialPowerSpectrum(lnAs,self.params_cosmo['n_s'])
        self.pk_in_primordial = self.initial_cosmo.getPrimordialPowerSpectrum()
        hitomipy.readInputPrimordialPowerSpectrum_py(self.k_in, self.pk_in_primordial
, len(self.k_in))

    ## normalization ##
    hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
    h = self.params_cosmo['h']
    Omega_b = self.params_cosmo['omega_b']/h**2
    Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm']/h**2
    Tcmb = 2.7255
    n_s = self.params_cosmo['n_s']
    hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

    # IR resummation damping term
    if type(Sigma2)==int:
        self.Sigma2 = Sigma2
        self.dSigma2 = Sigma2
    else:
        self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks) #* self.sigma8_norm**2
        self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks) #* self.sigma8_norm**2

    ## compute bispectra ##
    NDIM = 3
    NCOMP = len(self.k)
    if NCOMP > 1024:
        print("# of NCOMP should be <= 1024, otherwise results become zero.")
        return output_dict_ini

    AA = pycuba.Cuhre(
        self.Integrand_B, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
        )["results"]

    bk_temp = np.zeros((NCOMP))
    for i in range(NCOMP):
        bk_temp[i] = AA[i]["integral"]

    ## finalize parameters ##
    hitomipy.finalizeInputPowerSpectrum_py()

    #####

    ## sigma8 normalisation ##
    if self.integrand == 'tree': bk_out = bk_temp*self.sigma8_norm**4
    elif self.integrand == 'SN': bk_out = bk_temp*self.sigma8_norm**2
    elif self.integrand == 'PNG': bk_out = bk_temp*self.sigma8_norm**3

    ## output dict ##
    self.BK = {
        "kbin": self.k,
        "K": bk_out,
        "ell1": self.ell1,
        "ell2": self.ell2,
        "ELL": self.ELL,
    }

def calc_B_diag_k_vector(
    self,
    kvec, los=[0,0,1],

```

```

alpha_perp=1, alpha_parallel=1, b1=2, b2=0, bG2=0, c1=0, c2=0, knl=.3,
Pshot=0, Bshot=0,
ks=.05,
part = 'tree',
):
    """Computation of the diagonal part of the bispectrum for every k modes.
    -----
    kvec : list
        wavevectors at which compute the kernel.
    los : list
        line of sight.
    alpha_perp : float, optional
        perpendicular AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    alpha_parallel : float, optional
        parallel AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    b1 : float, optional
        linear galaxy bias (default is 2).
    b2 : float, optional
        second order galaxy bias (default is 0).
    bG2 : float, optional
        tidal galaxy bias (default is 0).
    c1 : float, optional
        FoG first counterterm (default is 0).
    c2 : float, optional
        FoG second counterterm (default is 0).
    knl : float, optional
        cutoff scale for FoG terms. (default is 0.3).
    Pshot : float, optional
        shotnoise term (in power and bispectrum). (default is 0).
    Bshot : float, optional
        shotnoise term (in bispectrum). (default is 0).
    ks : float, optional
        Separation scale for IR resummation (default is 0.05), no impact on
        the calculation for physical scales.
    """

    ## flags ##
    output_dict_ini = {
        "kvec_1": kvec,
        "K": np.zeros((len(kvec))),
    }

    ## initialization ##
    hitomipy.initializeInputPowerSpectrum_py()

    ## calc. wigner 3j symbols ##
    hitomipy.setWigner3j_py()

    ## read linear power spectrum ##
    hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
    hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

    ## normalization ##
    hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
    h = self.params_cosmo['h']
    Omega_b = self.params_cosmo['omega_b']/h**2
    Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm])/h**2
    Tcmb = 2.7255
    n_s = self.params_cosmo['n_s']
    hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

    # IR resummation damping term
    self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks)
    self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks)

    ## compute bispectra ##
    nb_k = len(kvec)

```

```

bk_temp = np.zeros((nb_k, nb_k))

for i in range(nb_k):
    for j in range(nb_k):
        k1 = kvec[i]
        k2 = kvec[j]
        k3 = -k1-k2
        if part == 'tree':
            bk_temp[i,j] = hitomipy.B_Tree_FoG_Damping_diag_k_vector_py( k1,
k2, k3, los,
                                alpha_perp, alpha_parallel,
                                self.f, b1, b2, bG2, c1, c2,
                                knl, self.Sigma2, self.dSigma2,
                                )
        if part == 'SN':
            bk_temp[i,j] = hitomipy.B_SN_FoG_Damping_diag_k_vector_py( k1, k2
, k3, los,
                                alpha_perp, alpha_parallel,
                                self.f, b1, c1, c2,
                                knl, Pshot, Bshot, self.Sigma2, self.dSigma2,
                                )

## finalize parameters ##
hitomipy.finalizeInputPowerSpectrum_py()

#####

## sigma8 normalisation ##
if part == 'tree': bk_out = bk_temp*self.sigma8_norm**4
elif part == 'SN': bk_out = bk_temp*self.sigma8_norm**2
elif part == 'PNG': bk_out = bk_temp*self.sigma8_norm**3

## output dict ##
self.BK = {
    "k": kvec,
    "K": bk_out,
}

def calc_B(
    self,
    k, ell1, ell2, ELL,
    alpha_perp=1, alpha_parallel=1, b1=2, b2=0, bG2=0, c1=0, c2=0, knl=.3,
    Pshot=0, Bshot=0,
    fnlloc=0, fnlequi=0, fnlortho=0,
    integrand='tree',
    ks=.05,
    f=False
):
    """Computation of the bispectrum.
    -----
    k : list
        wavevectors at which compute the kernel.
    ell1 : int
        first multipole of the tripOSH decomposition.
    ell2 : int
        second multipole of the tripOSH decomposition.
    ELL : int
        third multipole of the tripOSH decomposition, LOS anisotropies.
    alpha_perp : float, optional
        perpendicular AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    alpha_parallel : float, optional
        parallel AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    b1 : float, optional
        linear galaxy bias (default is 2).
    b2 : float, optional
        second order galaxy bias (default is 0).
    bG2 : float, optional

```

```

        tidal galaxy bias (default is 0).
c1 : float, optional
    FoG first counterterm (default is 0).
c2 : float, optional
    FoG second counterterm (default is 0).
knl : float, optional
    cutoff scale for FoG terms. (default is 0.3).
Pshot : float, optional
    shotnoise term (in power and bispectrum). (default is 0).
Bshot : float, optional
    shotnoise term (in bispectrum). (default is 0).
integrand : str, optional
    part of the bispectrum model to compute. 'tree' (default),
    'SN' (shot noise) or 'PNG' (primordial non gaussianity).
ks : float, optional
    Separation scale for IR resummation (default is 0.05), no impact on
    the calculation for physical scales.
"""

self.k = k
self.ell1 = ell1
self.ell2 = ell2
self.ELL = ELL
self.alpha_perp = alpha_perp
self.alpha_parallel = alpha_parallel
self.b1 = b1
self.b2 = b2
self.bG2 = bG2
self.c1 = c1
self.c2 = c2
self.knl = knl
self.fnlloc = fnlloc
self.fnlequi = fnlequi
self.fnlortho = fnlortho
self.Pshot = Pshot
self.Bshot = Bshot
self.integrand = integrand

if f: self.f = f

(kbin2_out, kbin1_out) = np.meshgrid(self.k, self.k)

## flags ##
output_dict_ini = {
    "kbin1": kbin1_out,
    "kbin2": kbin2_out,
    "K": np.zeros((len(self.k), len(self.k))),
    "ell1": self.ell1,
    "ell2": self.ell2,
    "ELL": self.ELL,
}

## initialization ##
hitomipy.initializeInputPowerSpectrum_py()

## calc. wigner 3j symbols ##
hitomipy.setWigner3j_py()

## read linear power spectrum ##
hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

## normalization ##
hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
h = self.params_cosmo['h']
Omega_b = self.params_cosmo['omega_b']/h**2
Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm'])/h**2
Tcmb = 2.7255
n_s = self.params_cosmo['n_s']

```

```

hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

# IR resummation damping term
self.Sigma2 = hitomipy.Sig2_py(self.rs_drag, ks)
self.dSigma2 = hitomipy.dSig2_py(self.rs_drag, ks)

## compute bispectra ##
NDIM = 3
NCOMP = len(self.k)
if NCOMP > 1024:
    print("# of NCOMP should be <= 1024, otherwise results become zero.")
    return output_dict_ini

AA = []
for i in range(NCOMP):
    print("k1 = ", self.k[i], "h/Mpc")
    self.kmag1 = self.k[i]
    AA.append(
        pycuba.Cuhre(
            self.Integrand_B, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
        )["results"]
    )

bk_temp = np.zeros((NCOMP, NCOMP))
for i in range(NCOMP):
    for j in range(NCOMP):
        bk_temp[i, j] = AA[i][j]["integral"]

## finalize parameters ##
hitomipy.finalizeInputPowerSpectrum_py()

#####

## sigma8 normalisation ##
if self.integrand == 'tree': bk_out = bk_temp*self.sigma8_norm**4
elif self.integrand == 'SN': bk_out = bk_temp*self.sigma8_norm**2
elif self.integrand == 'PNG': bk_out = bk_temp*self.sigma8_norm**3

## output dict ##
self.BK = {
    "kbin1": kbin1_out,
    "kbin2": kbin2_out,
    "K": bk_out,
    "ell1": self.ell1,
    "ell2": self.ell2,
    "ELL": self.ELL,
}

def Integrand_B(self, ndim, xx, ncomp, ff, userdata):
    """Call of the function to integrate for bispectrum model.
    -----
    No direct call of this function needed.
    """

    self.xx_in = np.zeros(ndim[0])
    self.ff_out = np.zeros(ncomp[0])

    for i in range(ndim[0]):
        self.xx_in[i] = xx[i]

    self.ndim = ndim[0]
    self.ncomp = ncomp[0]

    if self.integrand=='tree':
        if self.diag:
            hitomipy.integrand_B_FoG_Damping_diag_py(
                self.xx_in,
                self.ndim,
                self.ff_out,

```

```
        self.ncomp,
        self.k,
        len(self.k),
        self.ell1,
        self.ell2,
        self.ELL,
        self.alpha_perp,
        self.alpha_parallel,
        self.f,
        self.b1,
        self.b2,
        self.bG2,
        self.c1,
        self.c2,
        self.knl,
        self.Sigma2,
        self.dSigma2,
    )
else:
    hitomipy.integrand_B_FoG_Damping_py(
        self.xx_in,
        self.ndim,
        self.ff_out,
        self.ncomp,
        self.k,
        len(self.k),
        self.ell1,
        self.ell2,
        self.ELL,
        self.kmag1,
        self.alpha_perp,
        self.alpha_parallel,
        self.f,
        self.b1,
        self.b2,
        self.bG2,
        self.c1,
        self.c2,
        self.knl,
        self.Sigma2,
        self.dSigma2,
    )

elif self.integrand=='SN':
    if self.diag:
        hitomipy.integrand_B_SN_FoG_Damping_diag_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ell1,
            self.ell2,
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.c1,
            self.c2,
            self.knl,
            self.Pshot,
            self.Bshot,
            self.Sigma2,
            self.dSigma2,
        )
    else:
        hitomipy.integrand_B_SN_FoG_Damping_py(
            self.xx_in,
```

```

        self.ndim,
        self.ff_out,
        self.ncomp,
        self.k,
        len(self.k),
        self.ell1,
        self.ell2,
        self.ELL,
        self.kmag1,
        self.alpha_perp,
        self.alpha_parallel,
        self.f,
        self.b1,
        self.c1,
        self.c2,
        self.kn1,
        self.Pshot,
        self.Bshot,
        self.Sigma2,
        self.dSigma2,
    )

elif self.integrand=='PNG':
    if self.diag:
        hitomipy.integrand_B_PNG_Damping_diag_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ell1,
            self.ell2,
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.fnllloc,
            self.fnlequi,
            self.fnlortho,
            self.Sigma2,
            self.dSigma2,
        )

    for i in range(ncomp[0]):
        ff[i] = self.ff_out[i]

    return 0

def kernel_computation_Pk(self, kernel_name, k, ELL, integrand='tree',to_save=False):
    """Call the power spectrum model kernel.
    -----
    kernel_name : str
        bias parameters associated to the kernel to compute.
    k : list
        wavevenumbers at which compute the kernel.
    ELL : int
        multipole.
    to_save : str, optional
        If 'False' (default), kernel not saved.
    """

    self.kernel_name = kernel_name
    self.k = k
    self.ELL = ELL

    self.calc_K(integrand=integrand)

    if to_save:

```

```

        self.PK['params_cosmo'] = self.params_cosmo
        self.PK['z'] = self.z
        self.save(to_save)

def calc_K(
    self,
    integrand='tree',
    ks=.05,
):
    """Computation of the power spectrum spectrum kernel.
    -----
    integrand : str, optional
        part of the bispectrum model to compute. 'tree' (default),
        'lloop' or 'counterterm'.
    ks : float, optional
        Separation scale for IR resummation (default is 0.05), no impact on
        the calculation for physical scales.
    """

    self.integrand = integrand
    ## flags ##
    output_dict_ini = {
        "kbin": self.k,
        "K": np.zeros(len(self.k)),
        "ELL": self.ELL,
    }

    ## initialization ##
    hitomipy.initializeInputPowerSpectrum_py()

    ## read linear power spectrum ##
    hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
    hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

    ## normalization ##
    hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
    h = self.params_cosmo['h']
    Omega_b = self.params_cosmo['omega_b']/h**2
    Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm']/h**2
    Tcmb = 2.7255
    n_s = self.params_cosmo['n_s']
    hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

    ## IR resummation damping term ##
    self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks)
    self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks)

    ## compute power spectra ##
    NCOMP = len(self.k)
    if NCOMP > 1024:
        print("# of NCOMP should be <= 1024, otherwise results become zero.")
        return output_dict_ini

    if 'lloop' in self.integrand:
        NDIM = 4
        NNEW = 10000
        NMIN = 2
        FLATNESS = 50
        MAXEVAL = 10000
        AA = pycuba.Suave(self.Integrand_K_Pk, NDIM, NNEW, NMIN, FLATNESS, ncomp=NCOM
P, maxeval = MAXEVAL, verbose=0 | 4)["results"]

    else:
        NDIM = 2
        AA = pycuba.Cuhre(
            self.Integrand_K_Pk, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
        )["results"]

    pk_temp = np.zeros((NCOMP))

```



```
for i in range(NCOMP):
    if self.kernel_name=='b2_b2_norm':
        pk_temp[i] = AA[0]["integral"]
    else:
        pk_temp[i] = AA[i]["integral"]

## finalize parameters ##
hitomipy.finalizeInputPowerSpectrum_py()

#####

## sigma8 normalisation ##
if self.integrand == 'tree': pk_out = pk_temp*self.sigma8_norm**2
elif self.integrand == 'counterterm': pk_out = pk_temp*self.sigma8_norm**2
elif self.integrand == '1loop_22': pk_out = pk_temp*self.sigma8_norm**4
elif self.integrand == '1loop_13': pk_out = pk_temp*self.sigma8_norm**4

## output dict ##
self.PK = {
    "kbin": self.k,
    "K": pk_out,
    "ELL": self.ELL,
    "kernel": self.kernel_name
}

def Integrand_K_Pk(self, ndim, xx, ncomp, ff, userdata):
    """Call of the function to integrate for kernel part.
    -----
    No direct call of this function needed.
    """

    self.xx_in = np.zeros(ndim[0])
    self.ff_out = np.zeros(ncomp[0])

    for i in range(ndim[0]):
        self.xx_in[i] = xx[i]

    self.ndim = ndim[0]
    self.ncomp = ncomp[0]

    if self.integrand=='tree':
        hitomipy.integrand_P_Kernel_Tree_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.kernel_name
        )

    elif self.integrand=='counterterm':
        hitomipy.integrand_P_Kernel_Counterterm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.kernel_name
        )
```

```

elif self.integrand=='1loop_22':
    if self.kernel_name=='b2_b2_norm':
        hitomipy.integrand_P_Kernel_1loop_22_norm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            np.ones(1),
            1,
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
        )
    else:
        hitomipy.integrand_P_Kernel_1loop_22_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.f,
            self.Sigma2,
            self.dSigma2,
            self.kernel_name
        )

elif self.integrand=='1loop_13':
    hitomipy.integrand_P_Kernel_1loop_13_py(
        self.xx_in,
        self.ndim,
        self.ff_out,
        self.ncomp,
        self.k,
        len(self.k),
        self.ELL,
        self.f,
        self.Sigma2,
        self.dSigma2,
        self.kernel_name
    )

for i in range(ncomp[0]):
    ff[i] = self.ff_out[i]

return 0

def calc_P(
    self,
    k, ELL,
    alpha_perp=1, alpha_parallel=1, b1=2, b2=0, bG2=0, bGamma3=0,
    c0=0, c1=0, c2=0, ch=0, knl=.3,
    integrand='tree',
    ks=.05,
    real_space=False,
):
    """Computation of the power spectrum.
    -----
    k : list
        wavevenumbers at which compute the kernel.
    ELL : int
        third multipole of the tripoSH decomposition, LOS anisotropies.
    alpha_perp : float, optional
        perpendicular AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    alpha_parallel : float, optional
        parallel AP scaling between cosmo_fid and the cosmology studied

```

```

        default is 1.
    b1 : float, optional
        galaxy bias (default is 2).
    b2 : float, optional
        galaxy bias (default is 0).
    bG2 : float, optional
        galaxy bias (default is 0).
    bGamma3 : float, optional
        galaxy bias (default is 0).
    c0 : float, optional
        Counterterm (default is 0).
    c1 : float, optional
        FoG first counterterm (default is 0).
    c2 : float, optional
        FoG second counterterm (default is 0).
    ch : float, optional
        higher order counterterm (default is 0).
    knl : float, optional
        cutoff scale for FoG counterterms. (default is 0.3).
    Pshot : float, optional
        shotnoise term (in power and bispectrum). (default is 0).
    integrand : str, optional
        part of the power spectrum model to compute. `tree` (default),
        `lloop` or `counterterm`.
    ks : float, optional
        Separation scale for IR resummation (default is 0.05), no impact on
        the calculation for physical scales.
"""

self.k = k
self.ELL = ELL
self.alpha_perp = alpha_perp
self.alpha_parallel = alpha_parallel
self.b1 = b1
self.b2 = b2
self.bG2 = bG2
self.bGamma3 = bGamma3
self.c0 = c0
self.c1 = c1
self.c2 = c2
self.ch = ch
self.knl = knl
self.integrand = integrand

if real_space: self.f = 0

## flags ##
output_dict_ini = {
    "kbin": self.k,
    "K": np.zeros(len(self.k)),
    "ELL": self.ELL,
}

## initialization ##
hitomipy.initializeInputPowerSpectrum_py()

## read linear power spectrum ##
hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

## normalization ##
hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
h = self.params_cosmo['h']
Omega_b = self.params_cosmo['omega_b']/h**2
Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm'])/h**2
Tcmb = 2.7255
n_s = self.params_cosmo['n_s']
hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

```

```

    ## IR resummation damping term ##
    self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks)
    self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks)

    ## compute power spectra ##
    NCOMP = len(self.k)
    if NCOMP > 1024:
        print("# of NCOMP should be <= 1024, otherwise results become zero.")
        return output_dict_ini

    if 'lloop' in self.integrand:
        NDIM = 4
        NNEW = 10000
        NMIN = 2
        FLATNESS = 50
        MAXEVAL = 200000
        #AA = pycuba.Cuhre(
        #    self.Integrand_P, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
        #    )["results"]
        AA = pycuba.Suave(self.Integrand_P, NDIM, NNEW, NMIN, FLATNESS, ncomp=NCOMP,
maxeval = MAXEVAL, verbose=0 | 4)["results"]

    else:
        NDIM = 2
        AA = pycuba.Cuhre(
            self.Integrand_P, NDIM, ncomp=NCOMP, key=0, verbose=0 | 4
        )["results"]

    pk_temp = np.zeros((NCOMP))
    for i in range(NCOMP):
        pk_temp[i] = AA[i]["integral"]

    ## finalize parameters ##
    hitomipy.finalizeInputPowerSpectrum_py()

    #####

    ## sigma8 normalisation ##
    if 'tree' in self.integrand: pk_out = pk_temp*self.sigma8_norm**2
    elif 'counterterm' in self.integrand: pk_out = pk_temp*self.sigma8_norm**2
    elif 'lloop' in self.integrand: pk_out = pk_temp*self.sigma8_norm**4

    ## output dict ##
    self.PK = {
        "kbin": self.k,
        "K": pk_out,
        "ELL": self.ELL,
    }

def calc_P_k_vector(
    self,
    kvec, los=[0,0,1],
    alpha_perp=1, alpha_parallel=1, b1=2, b2=0, bG2=0, bGamma3=0,
    c0=0, c1=0, c2=0, ch=0, knl=.3,
    ks=.05, part='tree'
):
    """Computation of the power spectrum for every k modes.
    -----
    kvec : list
        wavevectors at which compute the kernel.
    los : list
        line of sight.
    alpha_perp : float, optional
        perpendicular AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    alpha_parallel : float, optional
        parallel AP scaling between cosmo_fid and the cosmology studied
        default is 1.
    b1 : float, optional
        linear galaxy bias (default is 2).

```

```

b2 : float, optional
    second order galaxy bias (default is 0).
bG2 : float, optional
    tidal galaxy bias (default is 0).
c0 : float, optional
    Counterterm (default is 0).
c1 : float, optional
    FoG first counterterm (default is 0).
c2 : float, optional
    FoG second counterterm (default is 0).
knl : float, optional
    cutoff scale for FoG terms. (default is 0.3).
ks : float, optional
    Separation scale for IR resummation (default is 0.05), no impact on
    the calculation for physical scales.
"""

## flags ##
output_dict_ini = {
    "kvec": kvec,
    "K": np.zeros((len(kvec))),
}

## initialization ##
hitomipy.initializeInputPowerSpectrum_py()

## calc. wigner 3j symbols ##
hitomipy.setWigner3j_py()

## read linear power spectrum ##
hitomipy.readInputPowerSpectrum_py(self.k_in, self.pk_in, len(self.k_in))
hitomipy.readInputNoWigglePowerSpectrum_py(self.k_in, self.pk_in_no_wiggle, len(s
elf.k_in))

## normalization ##
hitomipy.calcNormalizationUsingSigma8_py(self.sigma8_norm)
h = self.params_cosmo['h']
Omega_b = self.params_cosmo['omega_b']/h**2
Omega_m = (self.params_cosmo['omega_b']+self.params_cosmo['omega_cdm'])/h**2
Tcmb = 2.7255
n_s = self.params_cosmo['n_s']
hitomipy.calcNormalizationNoWiggle_py(1.0, h, Omega_b, Omega_m, Tcmb, n_s)

# IR resummation damping term
self.Sigma2 = hitomipy.Sig2_py(self.rs_drag,ks)
self.dSigma2 = hitomipy.dSig2_py(self.rs_drag,ks)

## compute bispectra ##
nb_k = len(kvec)

pk_temp = np.zeros(nb_k)

for i in range(nb_k):
    k = kvec[i]
    if part=='tree':
        pk_temp[i] = hitomipy.P_Tree_Damping_k_vector_py( k, los,
            alpha_perp, alpha_parallel,
            self.f, b1, self.Sigma2, self.dSigma2,
        )

    if part=='counterterm':
        pk_temp[i] = hitomipy.P_Counterterm_Damping_k_vector_py( k, los,
            alpha_perp, alpha_parallel,
            self.f, b1, c0, c1, c2, ch, knl, self.Sigma2, self.dSigma2,
        )

if part=='1loop':
    NDIM = 3
    NNEW = 10000

```

```

        NMIN = 2
        FLATNESS = 50
        MAXEVAL = 200000
        integrand = hitomipy.integrand_P_Damping_1loop_k_vector_py(
            np.zeros(NDIM),
            NDIM,
            np.zeros(nb_k),
            nb_k,
            kvec[:,0],
            kvec[:,1],
            kvec[:,2],
            nb_k,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.b2,
            self.bG2,
            self.bGamma3,
            self.Sigma2,
            self.dSigma2,
        )
        AA = pycuba.Suave(integrand, NDIM, NNEW, NMIN, FLATNESS, ncomp=NCOMP, maxeval
= MAXEVAL, verbose=0 | 4) ["results"]

        for i in range(nb_k):
            pk_temp[i] = AA[i] ["integral"]

    ## finalize parameters ##
    hitomipy.finalizeInputPowerSpectrum_py()

    #####

    ## sigma8 normalisation ##
    if part == 'tree': pk_out = pk_temp*self.sigma8_norm**2
    elif part == 'counterterm': pk_out = pk_temp*self.sigma8_norm**2
    elif part == '1loop': pk_out = pk_temp*self.sigma8_norm**4

    ## output dict ##
    self.PK = {
        "k": kvec,
        "K": pk_out,
    }

def Integrand_P(self, ndim, xx, ncomp, ff, userdata):
    """Call of the function to integrate for power spectrum model.
    -----
    No direct call of this function needed.
    """

    self.xx_in = np.zeros(ndim[0])
    self.ff_out = np.zeros(ncomp[0])

    for i in range(ndim[0]):
        self.xx_in[i] = xx[i]

    self.ndim = ndim[0]
    self.ncomp = ncomp[0]

    if self.integrand=='tree':
        hitomipy.integrand_P_Damping_Tree_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,

```

```
        self.alpha_perp,
        self.alpha_parallel,
        self.f,
        self.b1,
        self.Sigma2,
        self.dSigma2,
    )

    if self.integrand=='tree_for_1loop':
        hitomipy.integrand_P_Damping_Tree_for_1loop_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.Sigma2,
            self.dSigma2,
        )

    if self.integrand=='tree_noIR':
        hitomipy.integrand_P_Tree_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
        )

    if self.integrand=='tree_gm':
        hitomipy.integrand_P_Damping_Tree_gm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.Sigma2,
            self.dSigma2,
        )

    elif self.integrand=='counterterm_noIR':
        hitomipy.integrand_P_Counterterm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
```

```
        self.c0,
        self.c1,
        self.c2,
        self.kn1,
    )

    elif self.integrand=='counterterm':
        hitomipy.integrand_P_Damping_Counterterm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.c0,
            self.c1,
            self.c2,
            self.ch,
            self.kn1,
            self.Sigma2,
            self.dSigma2,
        )

    elif self.integrand=='counterterm_gm':
        hitomipy.integrand_P_Damping_Counterterm_gm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.c0,
            self.c1,
            self.c2,
            self.kn1,
            self.Sigma2,
            self.dSigma2,
        )

    elif self.integrand=='1loop_noIR':
        hitomipy.integrand_P_1loop_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.b2,
            self.bG2,
            self.bGamma3,
        )

    elif self.integrand=='1loop':
        hitomipy.integrand_P_Damping_1loop_py(
            self.xx_in,
```



```

        self.ndim,
        self.ff_out,
        self.ncomp,
        self.k,
        len(self.k),
        self.ELL,
        self.alpha_perp,
        self.alpha_parallel,
        self.f,
        self.b1,
        self.b2,
        self.bG2,
        self.bGamma3,
        self.Sigma2,
        self.dSigma2,
    )

    elif self.integrand=='lloop_gm':
        hitomipy.integrand_P_Damping_lloop_gm_py(
            self.xx_in,
            self.ndim,
            self.ff_out,
            self.ncomp,
            self.k,
            len(self.k),
            self.ELL,
            self.alpha_perp,
            self.alpha_parallel,
            self.f,
            self.b1,
            self.b2,
            self.bG2,
            self.bGamma3,
            self.Sigma2,
            self.dSigma2,
        )

    for i in range(ncomp[0]):
        ff[i] = self.ff_out[i]

    return 0

def calc_YYY(
    kvec1, kvec2, los=np.array([0,0,1], dtype='double'),
    ell1=0, ell2=0, ELL=0,
):
    """Computation of the normalized spherical harmonics function part of the bispectrum
    modeling.
    -----
    kvec1 : list
        wavevectors at which compute the function.
    kvec2 : list
        wavevectors at which compute the function.
    los : list
        line of sight.
    ell1 : int
        first multipole of the tripOSH decomposition.
    ell2 : int
        second multipole of the tripOSH decomposition.
    ELL : int
        third multipole of the tripOSH decomposition, LOS anisotropies.
    """

    return hitomipy.calcYYY_py(kvec1, kvec2, los,
                                ell1, ell2, ELL)

```